



B Submission Kit User Manual

Project: B Submission Kit

Repository: <https://github.com/TProver/B-Submission-Kit>

Audience: students using the plug-in, and teachers running the classroom server.

Provided by: [CLEARSY](https://clearsy.com) (Safety Solutions Designer).

Atelier B: <https://www.atelierb.eu>

1. Overview

BSK Submission - Classroom Dashboard					
Refreshes every 2 seconds. Last update: 2026-05-02T17:09:17Z. Students: 20.					
Expected projects: Airlock 10 submissions Tunnel 10 submissions					
Sort: Time (oldest first) Student Project Generate PDF report					
Student	Hostname	Project	Last submit	Overall	Stages
alice dupont last seen 2026-05-02T17:09:07Z	sim-host-00	Airlock	2026-05-02T17:09:05Z 1 300 B	verifying	typecheck: pending pog: pending prove: pending
alice dupont last seen 2026-05-02T17:09:07Z	sim-host-00	Tunnel	2026-05-02T17:09:07Z 2 429 B	queued	typecheck: pending pog: pending prove: pending
bob martin last seen 2026-05-02T17:09:15Z	sim-host-01	Airlock	2026-05-02T17:09:12Z 1 300 B	queued	typecheck: pending pog: pending prove: pending
bob martin last seen 2026-05-02T17:09:15Z	sim-host-01	Tunnel	2026-05-02T17:09:15Z 2 429 B	queued	typecheck: pending pog: pending prove: pending
claire lefevre last seen 2026-05-02T10:44:30Z	sim-host-02	Airlock	2026-05-02T10:42:51Z 1 301 B	partial	typecheck: ok pog: ok prove: ok animate: 4/5 animate: 01_smoke_test: ok 02_normal_user: ok 03_long_run: ok 04_init_only: ok 05_bad_op: fail typecheck:ok, pog:ok, prove:ok, animate:partial (4/5 passed; failed: 05_bad_op)
claire lefevre last seen 2026-05-02T10:44:30Z	sim-host-02	Tunnel	2026-05-02T10:42:52Z 2 445 B	ok	typecheck: ok pog: ok prove: ok animate: 2/2 animate: 01_smoke ok 02_traffic ok typecheck:ok, pog:ok, prove:ok, animate:ok (2/2 passed)

BSK lets a teacher run a classroom session in which students working in Atelier B submit their B projects to a verification server with one click. The server runs a four-stage pipeline (typecheck, proof-obligation generation, proof, ProB animation), aggregates the results on a live dashboard, and produces an archival PDF report on demand.

Roles:

- **Student:** installs a plug-in inside Atelier B once, then uses two menu entries during a session: *Connect* and *Submit and verify*.
- **Teacher:** installs the server on one machine on the classroom LAN, defines the expected projects and their animation scenarios, watches the dashboard live, generates a PDF report at the end of the session.

A single classroom server handles a cohort of up to ~50 students.

2. For students

2.1 Install the plug-in (once)

The teacher distributes the BSK plug-in folder (or a link to its installer). On Windows:

1. Open File Explorer in the `install/windows/` folder of the BSK distribution.
2. Right-click `install_plugin.cmd` → **Run as administrator**.
3. The installer copies six files (two `.etool` descriptors, the Python client, two PNG icons, and a small `.cmd` wrapper) into the Atelier B `share\plugins\` folder. By default the installer assumes Atelier B Community Edition 24.04.2 at `C:\Program Files\Atelier B Community Edition 24.04.2 24.04.2\`.
If your Atelier B version or install path differs (for example Pro 25.02, or a non-default location), set the `ATB_ROOT` environment variable before running the installer:

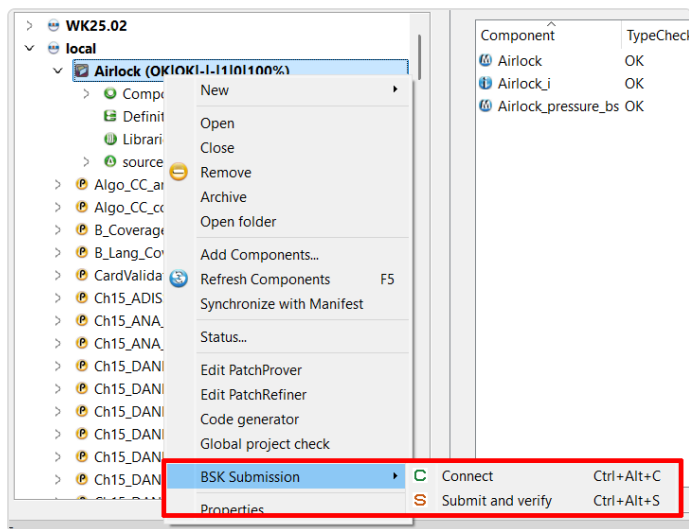
```
cmd set ATB_ROOT=C:\Program Files\Atelier B Professional Edition 25.02 25.02
install_plugin.cmd
```

The installer aborts with a clear error if the path does not contain a `share\` folder. 4. Close and reopen Atelier B. You should see a new **Project** → **BSK Submission** submenu with two entries: **Connect** and **Submit and verify**.

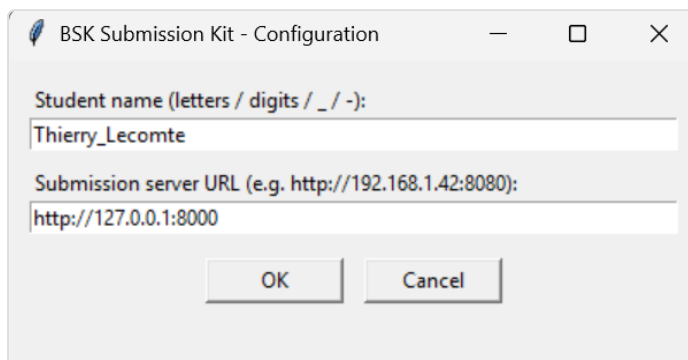
On Linux: run `install/linux/install_plugin.sh` with `sudo` (it does the equivalent copy into the Atelier B `share/plugins/` folder). The Linux installer auto-discovers `/opt/atelierb-*` and `/usr/share/atelierb` candidates; if your install is elsewhere, override with `sudo ATB_ROOT=/path/to/atelierb bash install_plugin.sh`.

You only need to do this once per machine.

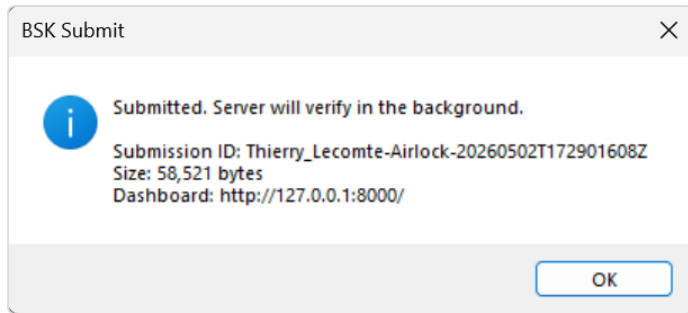
2.2 Connect for the first time



1. Open any B project in Atelier B (the menu items are inactive until a project is open).
2. **Project** → **BSK Submission** → **Connect** (shortcut `Ctrl+Alt+C`).
3. A small dialog asks for:
4. **Student name** (letters, digits, `_`, `-` ; this is what appears on the dashboard).
5. **Server URL** (the teacher tells you the address, e.g. `http://192.168.1.42:8000`).



1. Click **OK**. A confirmation popup appears with the dashboard URL, open it in your browser to see your row.



Behind the scenes, the server **claims your name for this Atelier B installation** and issues a short secret which the plug-in stores at `%APPDATA%\BSKSubmissionKit\config.json`. You will not be asked for it again unless you change machine or wipe that file.

2.3 Submit and verify

Once connected, every time you want the server to verify your project:

1. **Project** → **BSK Submission** → **Submit and verify** (shortcut `Ctrl+Alt+S`).
2. A small popup confirms the upload size and gives you the dashboard URL.
3. The server queues the verification. On the dashboard, the row for your project goes through `queued` → `verifying` → `ok` | `partial` | `fail` .

You can submit as many times as you like during the session; the server keeps only the latest submission per (you, project) pair.

2.4 Reading the dashboard

Open the dashboard URL in any browser. Refreshes every two seconds.

Column	Meaning
Student	Your name. Click to download the full bbatch transcript for the latest submission of that project.
Hostname	The machine that submitted.
Project	The Atelier B project name.
Last submit	UTC timestamp + zip size.
Overall	One of three coloured badges.
Stages	Per-stage pills + a second row of per-scenario chips for animate.

Overall verdicts:

Badge	Meaning
ok (green)	every stage of the pipeline succeeded
partial (amber)	some stages succeeded, some did not (typical when a model proves cleanly but one scenario diverges)
fail (red)	nothing succeeded (usually a typecheck or POG error early in the pipeline)

Stage pills follow the same colours, plus `skipped` (italic grey) when an earlier stage prevented a later one from running. Hover any pill or scenario chip to see a tooltip; click it to open the full per-stage log in a new tab.

Sort buttons at the top of the dashboard let you reorder rows by **Time (oldest first)** (the default), by **Student**, or by **Project**. Your choice persists across page reloads.

The **Expected projects** banner above the table lists the projects the teacher has configured for this session. If you submit something with a different project name, your row will still appear but no scenarios will run.

BSK Submission - Classroom Dashboard

Refreshes every 2 seconds. Last update: 2026-05-02T17:44:18Z. Students: 21.

Expected projects:
Airlock 11 submissions
Tunnel 10 submissions

Example of a row in the `partial` state, with per-scenario chips visible:

Overall	Stages
partial	typecheck: ok pog: ok prove: ok animate: 4/5 animate: 01_smoke_test: ok 02_normal_use: ok 03_long_run: ok 04_init_only: ok 05_bad_op: fail typecheck:ok, pog:ok, prove:ok, animate:partial (4/5 passed; failed: 05_bad_op)

2.5 Common errors and what to do

Symptom	Cause	Fix
Cannot run <code>BSKConnect</code> in Atelier B's <i>Errors</i> panel	Atelier B can't find Python	Set <code>BSK_PYTHON</code> env var to your <code>python.exe</code> path, or install Python 3.9+ from python.org with "Add to PATH" checked, then restart Atelier B
Failed to reach <code>http://... popup</code>	Server is not running, or wrong	Ask the teacher to confirm the URL; verify the LAN connection

Symptom	Cause	Fix
	URL	
Name 'alice' is already taken	Another machine already claimed that name	Pick a different name (e.g. <code>alice_2</code>) and run Connect again
Invalid session, re-run Connect on Submit	Token expired or server was restarted	Run Project → BSK Submission → Connect again, then re-Submit
Dashboard shows your row but the verdict is <code>fail</code> with <code>bbatch:fail</code>	Your model has a typecheck error	Open the row's <code>summary.log</code> link to see the bbatch error message; fix the model and resubmit

3. For teachers

3.1 Start the classroom server

Once per session:

```
cd c:\Work\PROJECTS\CLAUDE\D06-BSubmissionKit\receiver
start_server.cmd
```

First run installs FastAPI, Uvicorn and a few transitive dependencies into a local `.venv/` . Subsequent runs skip the install and start the server in ~1 s. The dashboard becomes reachable on `http://<your-LAN-IP>:8000/` . Tell students that URL.

Useful flags:

Command	What it does
<code>start_server.cmd</code>	Default (port 8000, keep all existing submissions)
<code>start_server.cmd 9000</code>	Port 9000 instead
<code>start_server.cmd --clean</code>	Move existing <code>submissions/</code> and <code>server_workspace/</code> into <code>archives/<timestamp>/</code> after a y/N confirmation, then start fresh
<code>start_server.cmd --purge</code>	DELETE existing submissions (irreversible; needs typing <code>DELETE</code> at the prompt)

Command	What it does
<code>start_server.cmd --clean 9000</code>	Combine: archive then start on 9000

Equivalent on Linux: `bash receiver/start_server.sh` with the same flags.

3.2 Define the projects

The teacher chooses the project names that students must use. For each project, drop a directory under `receiver/scenarios/`. The *expected projects banner* on the dashboard lists exactly those directory names.

```
receiver/scenarios/
├─ Airlock/
│   ├─ 01_smoke_test.scenario
│   ├─ 02_normal_use.scenario
│   ├─ 03_long_run.scenario
│   ├─ 04_init_only.scenario
│   └─ 05_bad_op.scenario
└─ Tunnel/
    ├─ 01_smoke.scenario
    └─ 02_traffic.scenario
```

Up to 5 scenarios per project (extra files are ignored). Each scenario file's name (without extension) is what appears on the dashboard's per-scenario chips.

3.3 Scenario format

Plain text, one operation call per line. Lines starting with `#` are comments; blank lines are ignored.

Recognised forms:

```
# initialise the machine (no parameters)
init

# initialise the machine with parameters
init(0)

# CONCRETE_CONSTANTS (required first line if the machine has any)
setup_constants(100)

# fire an operation with no parameters
car_enter

# fire an operation with input parameters (int, bool, or verbatim ProB term)
```

```

op_pow(3)
do(3, TRUE)
exotic(avl_set(node(a, b)), 3)

# fire an operation and ASSERT its return value(s)
get_int -> 5
op_pow(3) -> 8
get_all_values -> 7, 3

```

Without `->` the operation must just fire successfully; with `->` ProB additionally checks that the returned values match.

The converter expands these into ProB Prolog-format trace files on the fly and runs `probccli -trace_replay prolog`. Per-scenario timeout is 30 s (covers ProB's first-load SICStus warm-up).

3.4 Watch the dashboard live

Open the dashboard URL on the teacher PC (and project it to the class screen if you like). It refreshes every two seconds. Rows appear in the order students submit; use the **Sort** buttons to switch between Time / Student / Project orderings.

Per-(student × project) row counts: with 10 students × 2 projects = 20 rows.

Each cell is interactive:

- The **student name** opens the full bbatch transcript for that submission.
- Each **stage pill** opens the per-stage log.
- Each **scenario chip** under animate opens that scenario's full probcli output.

3.5 Generate the PDF report

Click **Generate PDF report** in the dashboard's sort bar. The server: 1. Builds a self-contained HTML document with the per-(student × project) summary table at the top, plus every embedded log (the same content the dashboard's clickable links reveal). 2. Renders it to PDF via headless Microsoft Edge. 3. Saves it under `reports/<projects>_YYYYMMDDTHMMSS.pdf` and opens it in a new tab.

`<projects>` is the alphabetical join of the project names submitted so far, truncated to 20 characters. Example file names:

- `Airlock_Tunnel_20260502T142315.pdf`
- `Airlock_Tunnel_Boiler_20260502T160500.pdf` (truncated if longer)

The PDF is the canonical archival artefact for grading and proof of verifications.

 First page of a generated PDF report

3.6 Manage student names

The first-come-first-serve scheme means a name cannot be claimed by two different machines:

- A student installed the plug-in and claimed `alice` from machine A.
- The same student then tries to **Connect** as `alice` from machine B → server returns HTTP 409 with the message *"Name 'alice' is already taken on this server. Choose another name and Connect again."*

If a student legitimately needs to recover their name (lost the plug-in config, reinstalled Atelier B), the teacher can release it manually:

1. Stop the server (`Ctrl+C` in the terminal).
2. Edit `submissions/_state.json` . Find the entry for the student id and remove its `"secret": "..."` line.
3. Restart `start_server.cmd` . The student can now re-Connect from any machine.

Alternatively, the student picks a new name (e.g. `alice2`), which is simpler.

3.7 End-of-session housekeeping

At the end of a teaching session:

1. (Optional) Click **Generate PDF report** one last time and archive the PDF wherever you keep grade records.
2. Stop the server (`Ctrl+C`).
3. For the next session, run `start_server.cmd --clean` so the previous cohort's data is moved into `archives/<timestamp>/` and the dashboard starts empty.

`archives/` is never auto-deleted; review and remove old archive folders manually when you no longer need them.

4. Reference

4.1 Scenario keywords

Keyword	Maps to	Notes
<code>init / initialise_machine</code>	<code>'\$initialise_machine'.</code>	Required if the machine has any state.
<code>init(args)</code>	<code>'\$initialise_machine'(args).</code>	If INITIALISATION takes parameters.
<code>setup_constants(N1, N2, ...)</code>	<code>'\$setup_constants'(int(N1), int(N2), ...).</code>	Required first line for any machine with <code>CONCRETE_CONSTANTS</code> . Without it, ProB rejects trace replay.

Keyword	Maps to	Notes
<code>op</code>	<code>'op'.</code>	No-arg, no-return.
<code>op(3, TRUE)</code>	<code>'op'(int(3), bool_true).</code>	Scalar args: integers, booleans (TRUE/FALSE/T/F), or verbatim ProB terms.
<code>op -> 5</code>	<code>'op'-->[int(5)].</code>	Asserts the return value.
<code>op(3) -> 8, FALSE</code>	<code>'op'(int(3))-->[int(8), bool_false].</code>	Multiple inputs, multiple returns.

4.2 Server CLI flags and environment variables

```

start_server.cmd [PORT]           # default port 8000
start_server.cmd --clean [PORT]    # archive submissions/ + server_workspace/, then start
start_server.cmd --purge [PORT]    # DELETE submissions/ + server_workspace/ (DELETE-confirm)

```

Env var	Default	Purpose
<code>BSK_BBATCH</code>	<code>C:\Program Files\Atelier B Community Edition 24.04.2 24.04.2\bin\bbatch.exe</code>	Atelier B <code>bbatch</code> binary. Override when running a different version (e.g. Pro 25.02 or Linux): <code>set BSK_BBATCH=<path></code> before launching <code>start_server.cmd</code> .
<code>BSK_PROBCLI</code>	<code>C:\Tools\ProB\probcli.exe</code>	ProB CLI binary
<code>BSK_EDGE</code>	<code>C:\Program Files (x86)\Microsoft\Edge\Application\msedge.exe</code>	Headless Edge for PDF report rendering
<code>BSK_PYTHON</code>	discovered via PATH	Python 3 interpreter for the plug-in client (student side)
<code>BSK_WORKSPACE</code>	<code><project>/server_workspace</code>	Directory <code>bbatch</code> uses for its per-submission projects
<code>BSK_SCENARIOS</code>	<code><project>/receiver/scenarios</code>	Scenarios folder root

4.3 Files on disk

Path	Purpose	Lifecycle
<code>submissions/_state.json</code>	In-memory state mirror; JSON snapshot rewritten after every change	Created on first submit; survives server restart
<code>submissions/<sid>/_incoming/<id>.zip</code>	Submit handler stages new uploads here	Moved into <code>_work_<id>/</code> by the worker, then removed
<code>submissions/<sid>/_work_<id>/</code>	Per-submission staging area for verification	Renamed to <code>latest_<project>/</code> when verify finishes
<code>submissions/<sid>/latest_<project>/</code>	Most-recent verification result per (student, project)	Atomic-rename target; consulted by dashboard log links
<code>server_workspace/</code>	bbatch project workspace, one folder per submission	Accumulates; reset by <code>--clean</code>
<code>archives/<ts>/</code>	Previous sessions' data, preserved	Manual cleanup when no longer needed
<code>logs/server.log</code>	Rotating log of every submit/verify/swap event + worker tracebacks	Rotates at 2 MB, keeps 4 backups
<code>reports/<projects>_<ts>.pdf</code>	Generated archival reports	Manually move to grade records
<code>receiver/scenarios/<Project>/</code>	Teacher-defined animation scenarios	Edited offline between sessions

4.4 Where to ask for help

If something behaves unexpectedly, the first place to look is `logs/server.log`. Every submit, verify and worker step is logged with timestamp, student id, project name, and full stack trace on errors. Per-submission logs also live in `submissions/<sid>/latest_<project>/verification/`.